Sequence Diagrams

IS26-AM10

Contents

1 Login

1.1 Socket

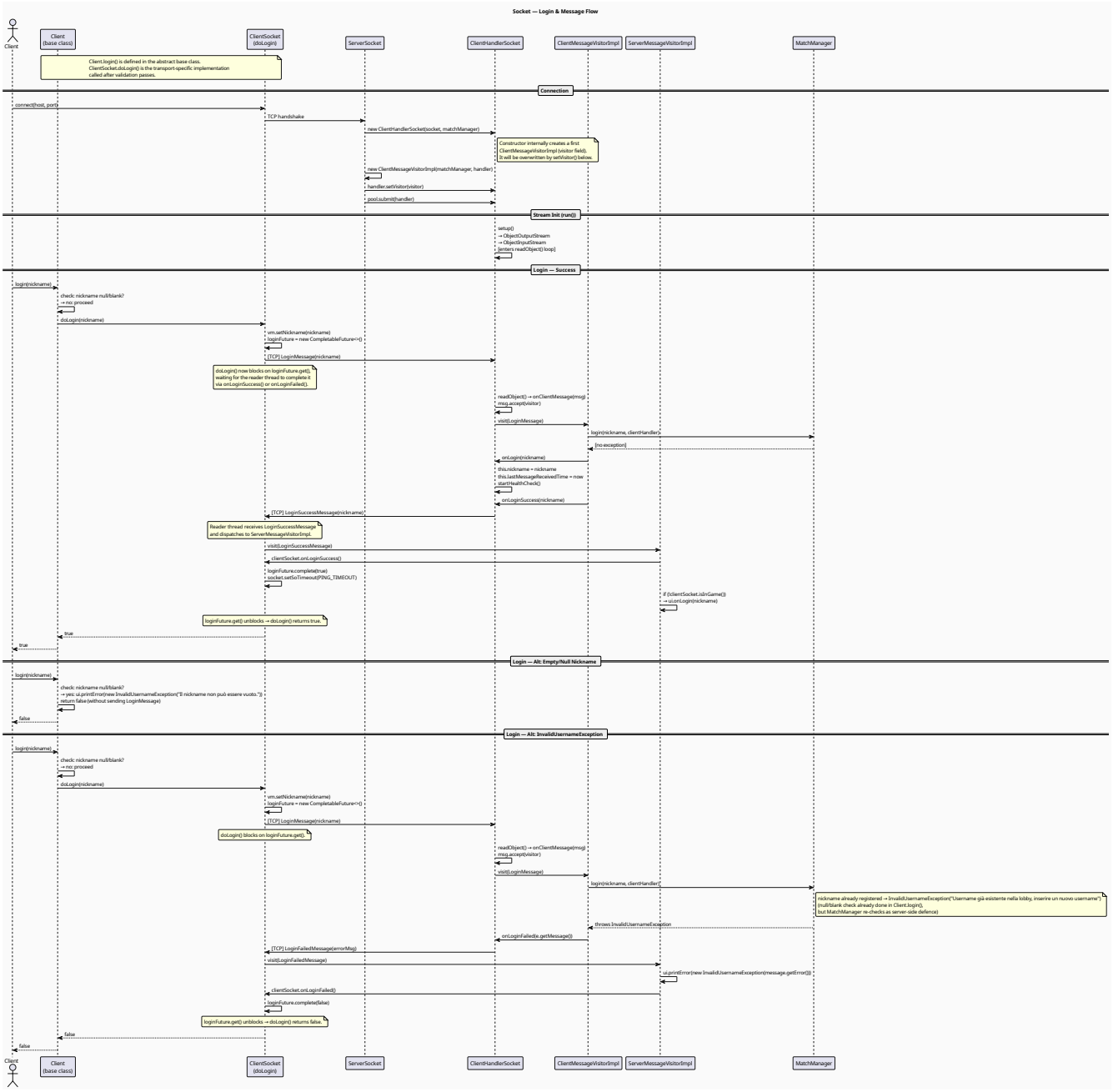

Figure 1: Sequence diagram for a Socket login

Covers the Socket login flow across three cases: successful login, client-side rejection of a null or blank nickname, and server-side `InvalidUsernameException`. Unlike RMI, the Socket client sends a `LoginMessage` and then blocks on a `CompletableFuture` until the reader thread completes it upon receiving either a `LoginSuccessMessage` or `LoginFailedMessage`. On success, the handler sets its nickname, starts the health check, and the client unblocks, sets a socket timeout, and triggers the UI. On failure, the future is completed with false and no health check is started.

1.2 RMI

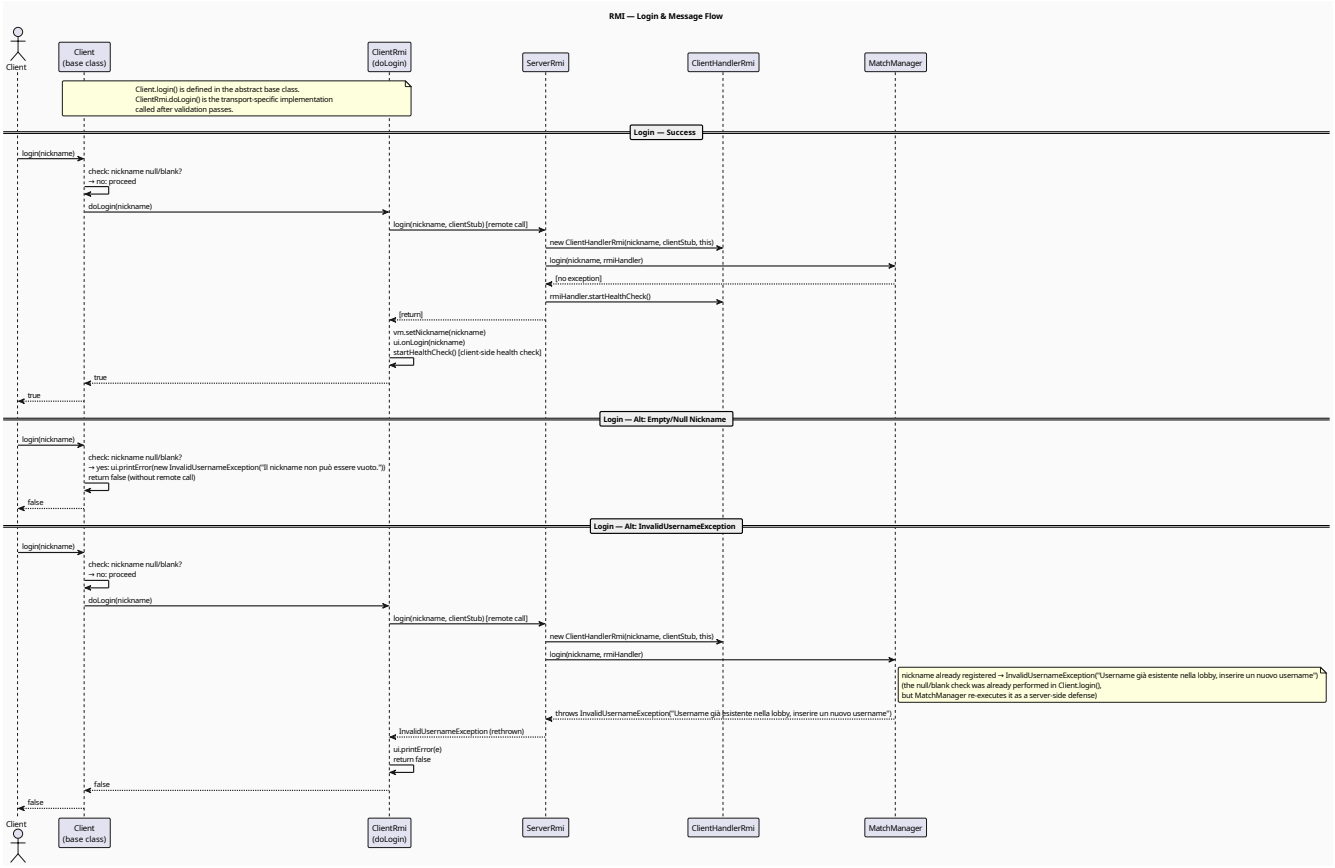

Figure 2: Sequence diagram for a RMI login

Covers the RMI login flow across three cases: successful login, client-side rejection of a null or blank nickname before any remote call is made, and server-side `InvalidUsernameException` when the nickname is already registered. On success, the server creates a `ClientHandlerRmi`, registers the player in `MatchManager`, and starts the health check on both sides. On failure, the handler is discarded and the health check is never started.

2 Healthcheck

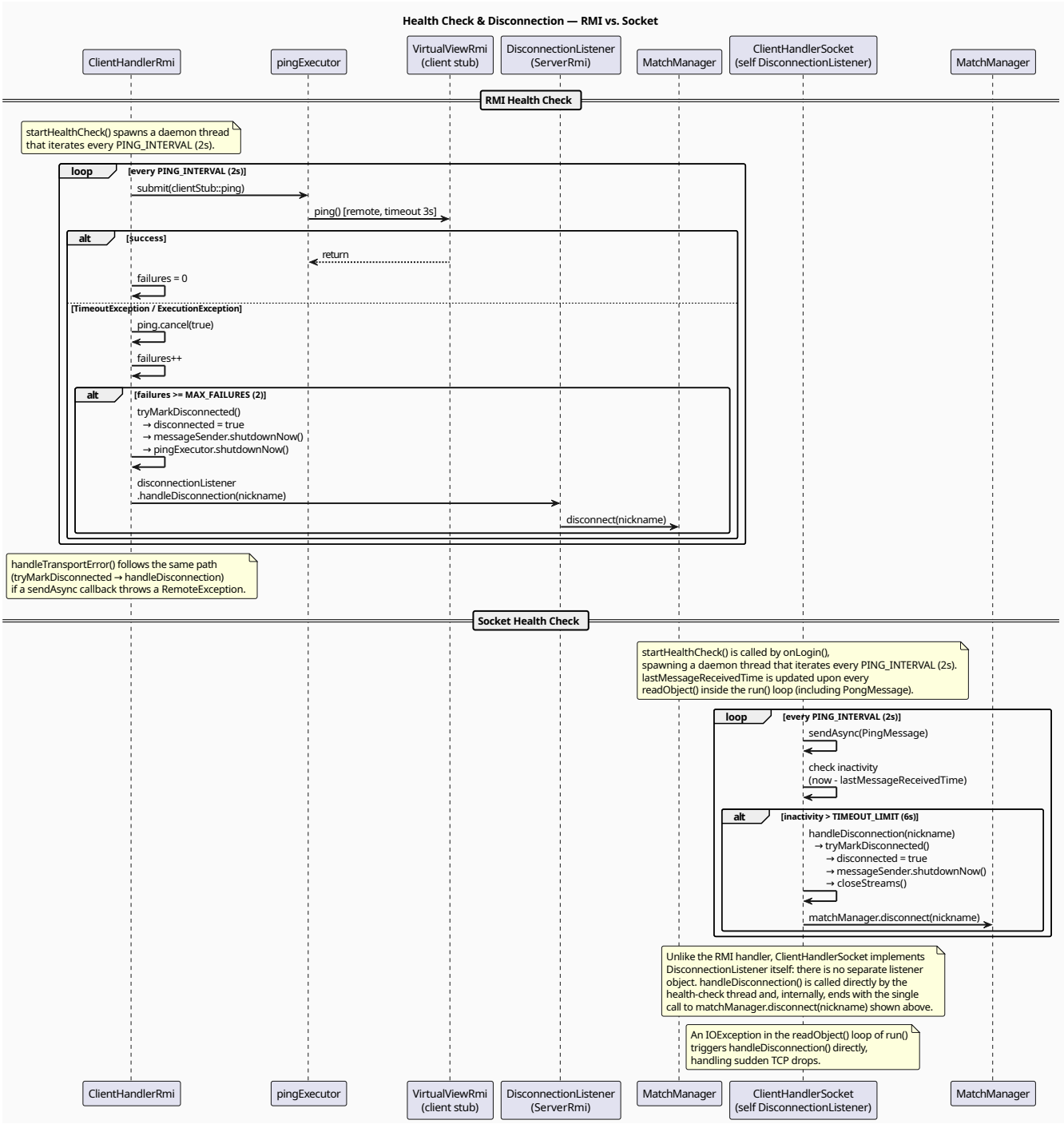

Figure 3: Sequence diagram to ping a client

Covers the periodic ping mechanisms used by both protocols to detect client disconnections. For RMI, `ClientHandlerRmi` submits timed ping calls to the client stub every 2 seconds, incrementing a failure counter on timeout until `MAXFAILURES` is reached, at which point the handler marks itself disconnected and notifies `MatchManager`. For Socket, `ClientHandlerSocket` sends `PingMessages` at the same interval and tracks the timestamp of the last received message, triggering disconnection if inactivity exceeds 6 seconds. Both paths converge on the same `matchManager.disconnect(nickname)` call, but differ structurally: RMI delegates to a separate `DisconnectionListener`, while the Socket handler implements the listener itself.

3 Lobby creation or join

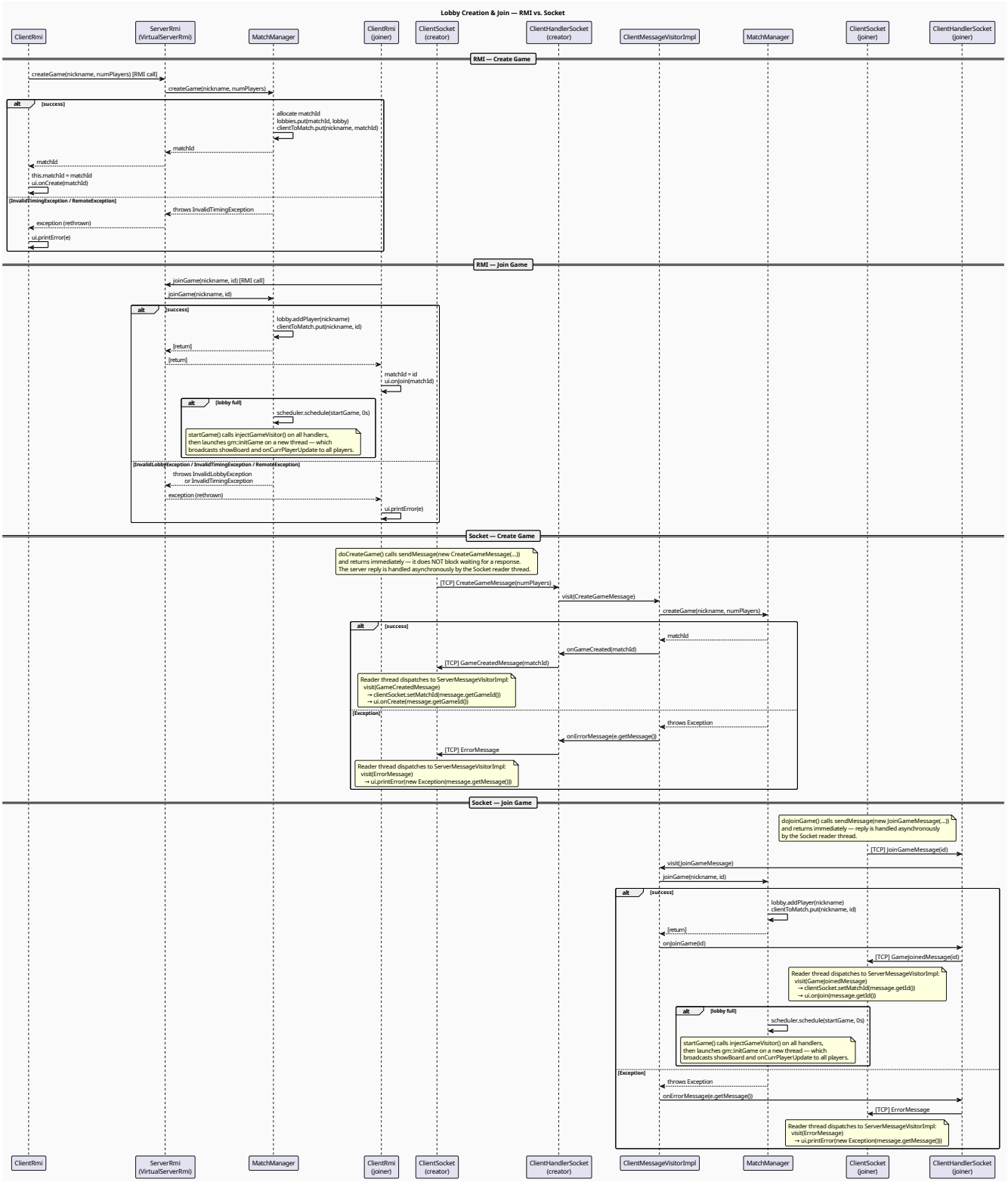

Figure 4: Sequence diagram representing how to create or join a lobby

Covers the create and join flows for both protocols. For RMI, both calls are synchronous and return the match ID directly to the client. For Socket, both calls are fire-and-forget: the client sends the message and handles the server reply asynchronously on the reader thread. In both protocols, a successful join that fills the lobby triggers a scheduled `startGame()`, which injects the game visitor into all handlers and broadcasts the initial board state to all players.

4 Game start and end

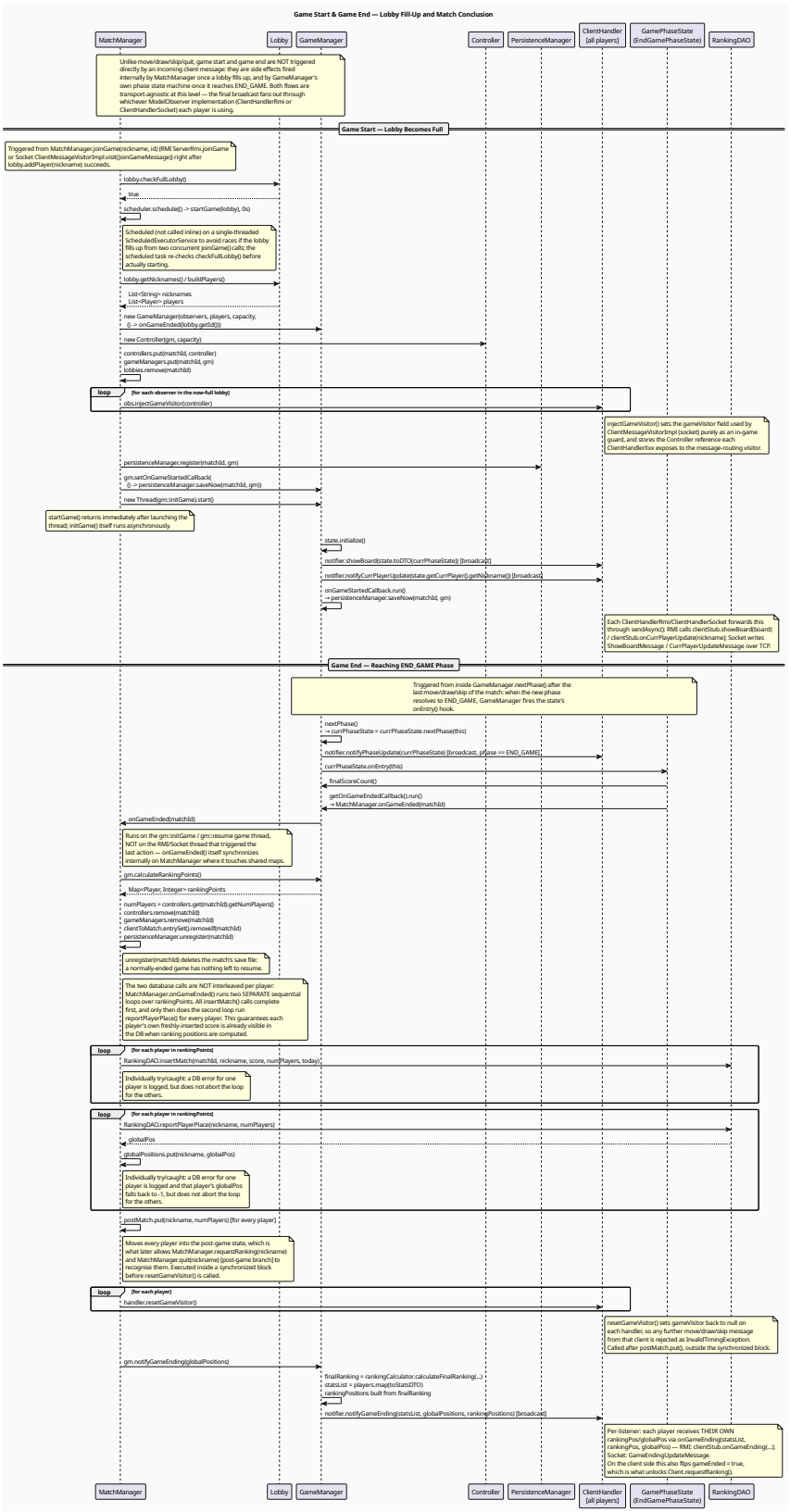

Figure 5: Sequence diagram showing what happens when a game starts and when a game ends

Covers the two server-side lifecycle events that are not triggered by direct client messages: the automatic game start when a lobby fills up, and the game end when the phase state machine reaches `ENDGAME`. The start flow shows how `MatchManager` schedules the game launch, initializes `GameManager` and `Controller`, injects the game visitor into all handlers, and broadcasts the initial board state. The end flow shows how `EndGamePhaseState` triggers score calculation, sequential database writes (match insertion before ranking reporting), cleanup of shared maps and save files, and the final per-player game-ending broadcast.

5 Available actions

5.1 Socket

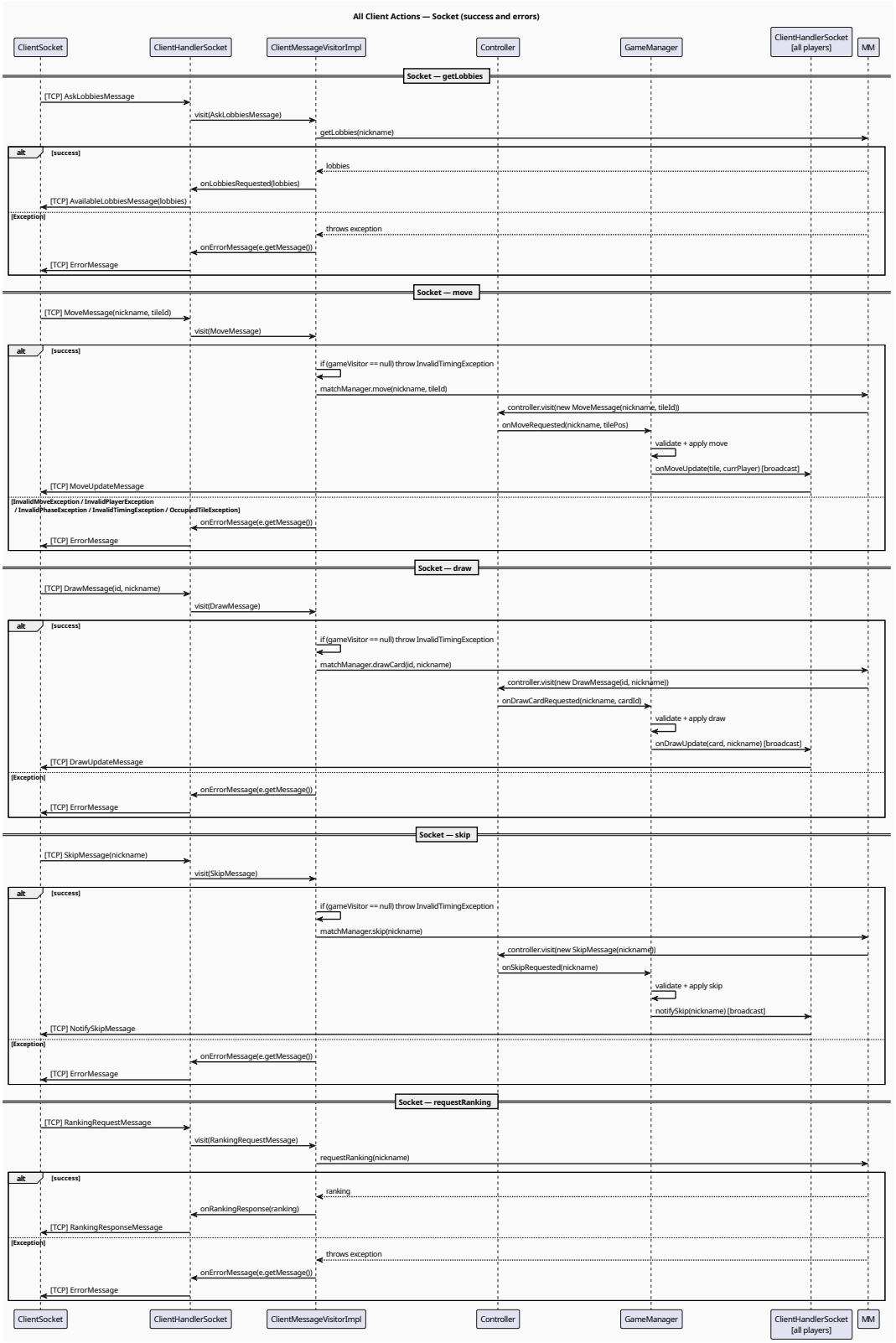

Figure 6: Sequence diagrams showing the actions that a client can perform

5.2 RMI

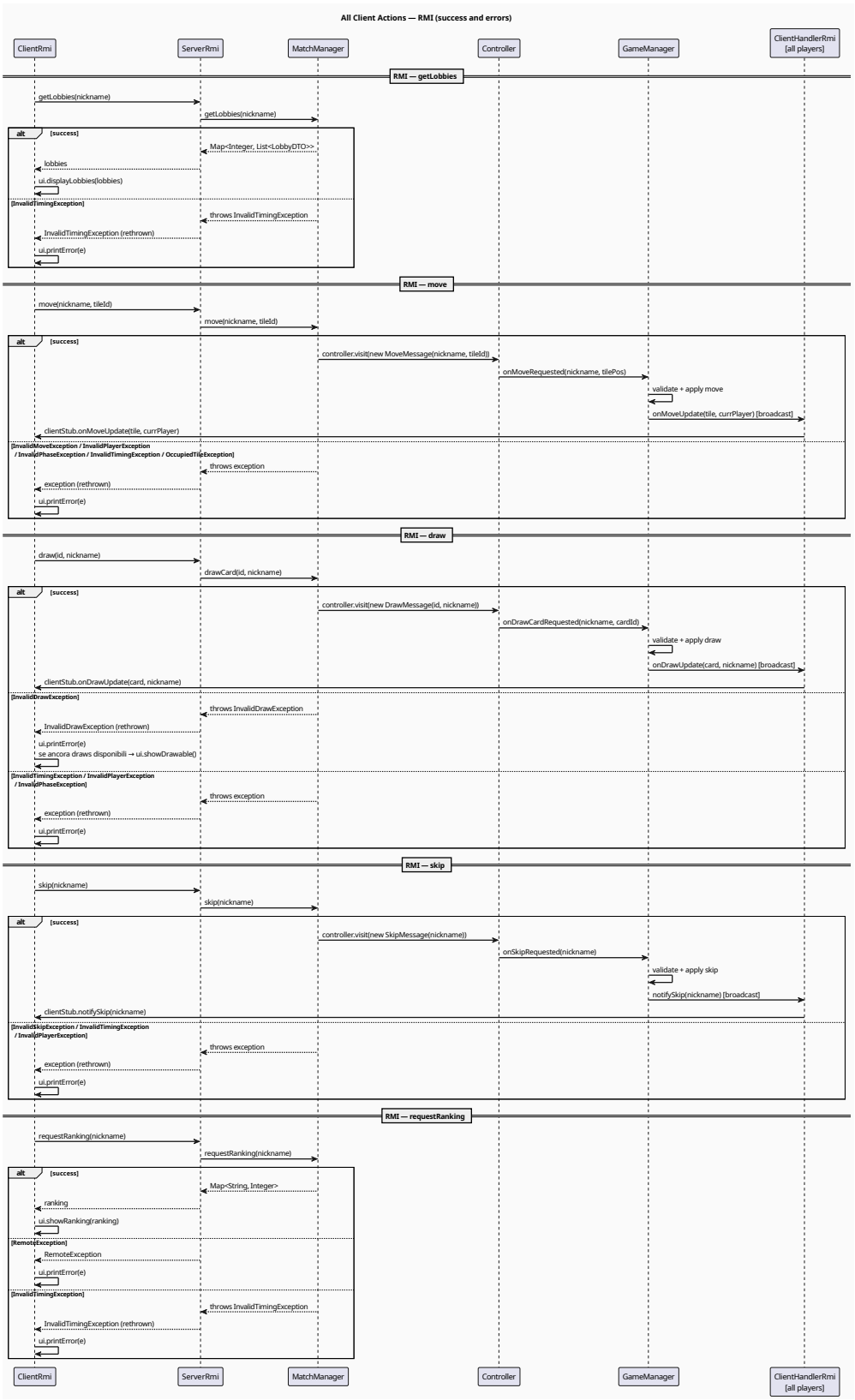

Figure 7: Sequence diagrams showing the actions that a client can perform

Covers every client-initiated action (`getLobbies`, `move`, `draw`, `skip`, `quit`, `requestRanking`) for both RMI and Socket protocols. For each action, the diagram shows the full call chain from client to `GameManager` and the broadcast back to all connected clients, alongside all possible error paths.

6 Quit

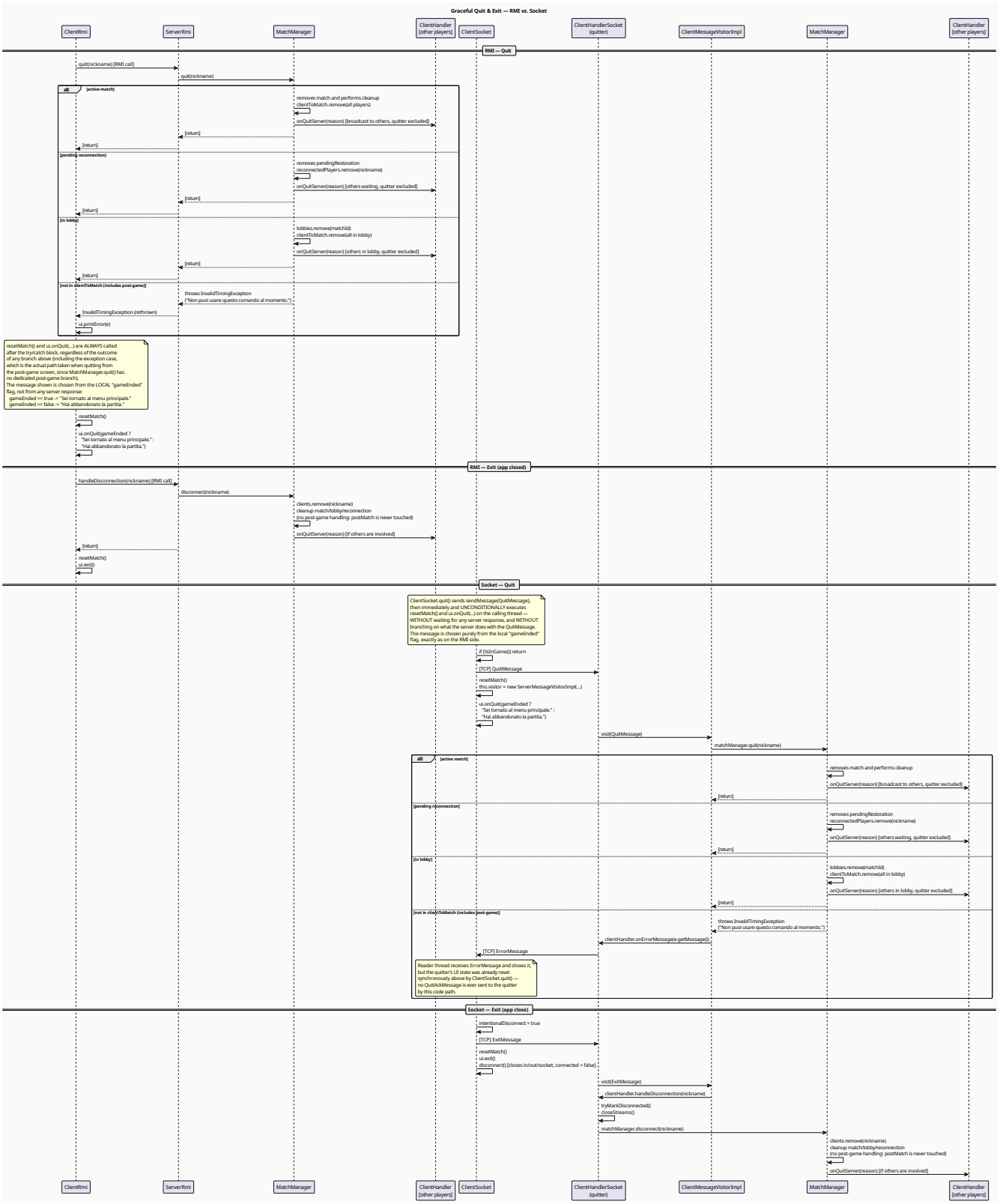

Figure 8: Sequence diagram showing what happens when a client quits or the app is closed

Covers voluntary quit and application exit for both protocols. Quit is tested against three server-recognized states (active match, pending reconnection, in lobby); a fourth case, including the post-game screen, which `MatchManager.quit()` has no dedicated branch for, falls through to an `InvalidTimingException`, silently caught by the client.

For RMI, quit is synchronous and cleanup always runs after the try/catch regardless of the server outcome. For Socket, quit is fire-and-forget: the client resets its local state immediately without waiting for the server. In both cases, the message shown to the user ("returned to menu" vs "abandoned match") is chosen locally from a `gameEnded` flag, not from the server's response. The exit flow follows the disconnection path in both protocols, delegating to `matchManager.disconnect()` and notifying any remaining players.

7 Transport Error

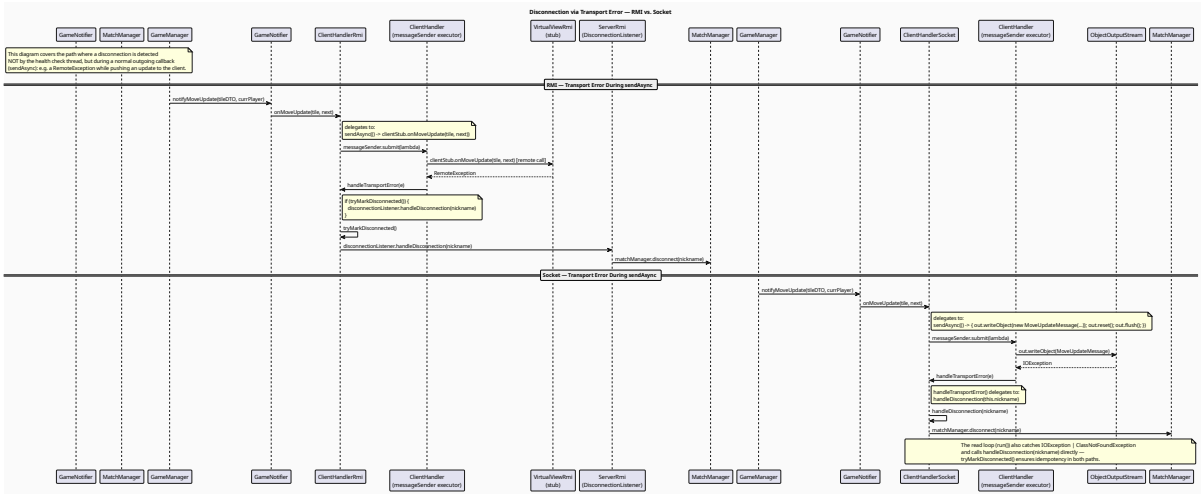

Figure 9: Sequence diagram showing what happens when a message is not sent correctly

Covers the disconnection path triggered not by the health check, but by a transport error occurring during a `sendAsync` outgoing callback. For RMI, a `RemoteException` from the client stub causes the `messageSender` executor to invoke `handleTransportError()`, which calls `tryMarkDisconnected()` and delegates to the `DisconnectionListener`. For Socket, an `IOException` on the `ObjectOutputStream` follows the same pattern via `handleDisconnection()` directly. In both cases, `tryMarkDisconnected()` acts as an idempotency guard, ensuring that concurrent triggers (health check and transport error) result in only one `disconnect()` call to `MatchManager`.

8 Client Disconnection and Server Crash

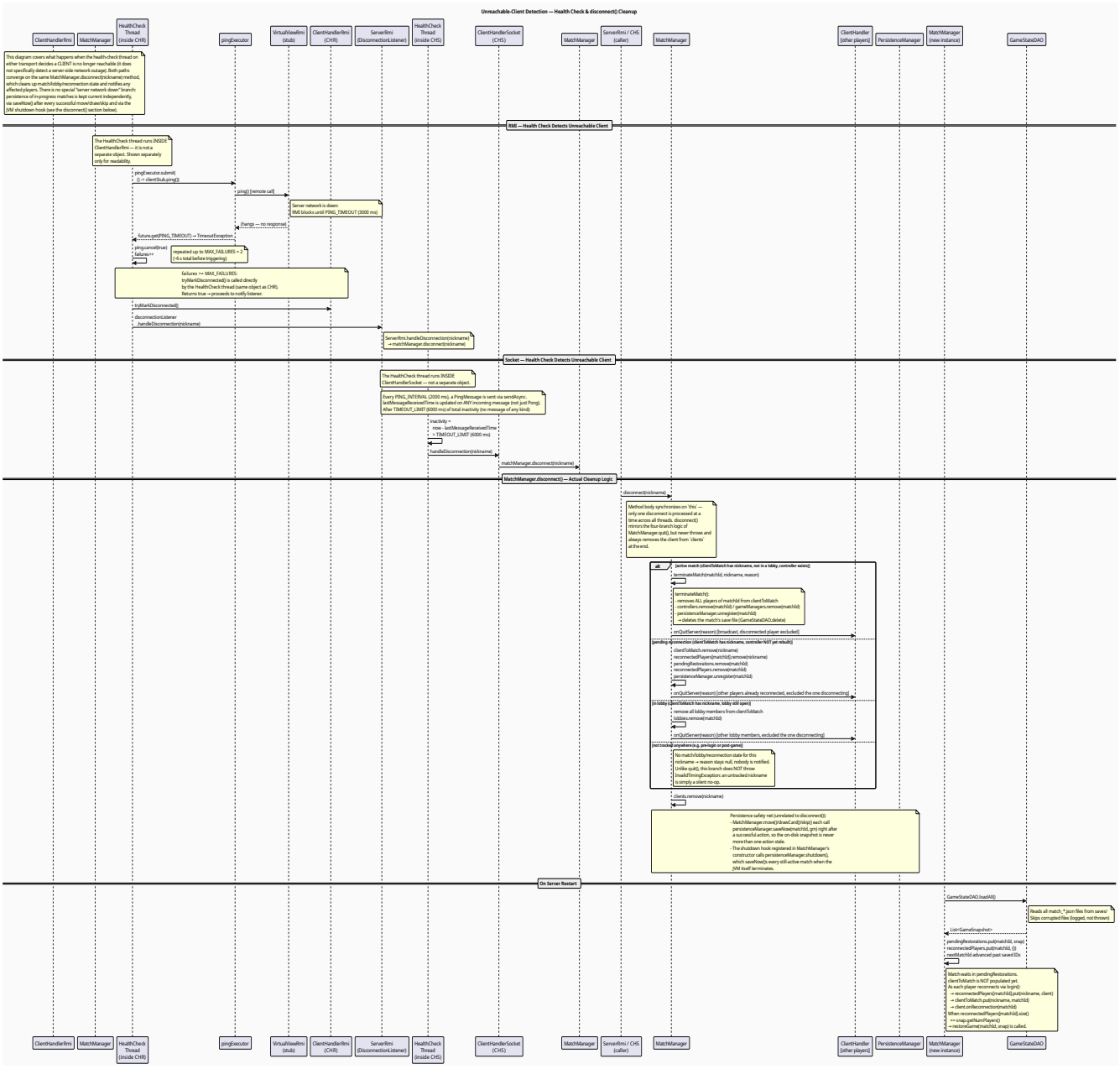

Figure 10: Sequence diagram showing what happens when the server crashes

Covers what happens when the health-check thread determines a client is no longer reachable, for both RMI (ping timeout after `MAXFAILURES` attempts) and Socket (inactivity exceeding `TIMEOUT_LIMIT`). Both paths converge on `MatchManager.disconnect()`, which handles four cases (active match, pending reconnection, in lobby, and untracked), cleaning up shared state and notifying remaining players accordingly.

The diagram also documents the persistence safety net (`saveNow()` after every action and on JVM shutdown) and the server restart flow, where persisted snapshots are loaded into `pendingRestorations` and matches are restored incrementally as players reconnect.

9 Reconnection

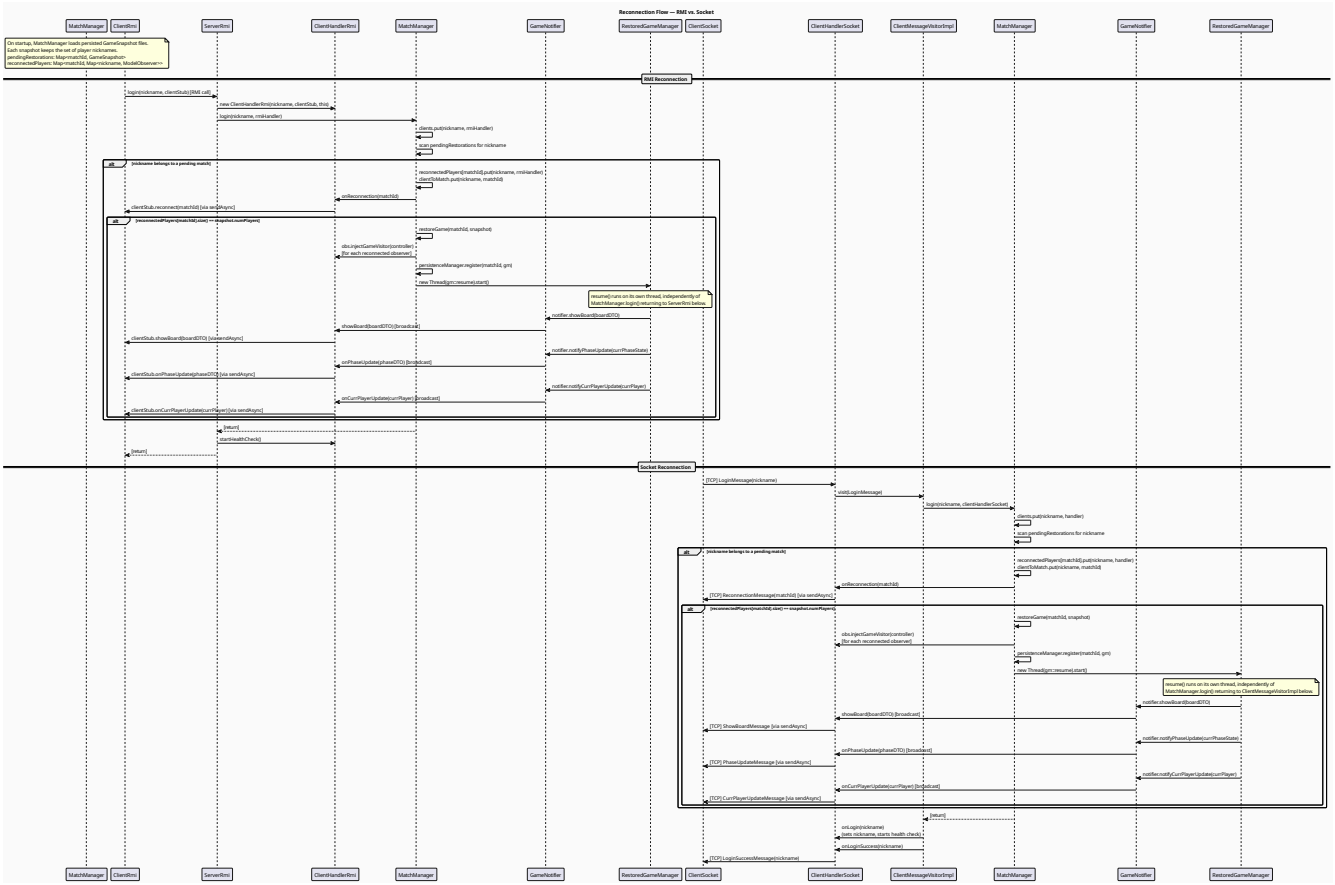

Figure 11: Sequence diagrams showing how clients can reconnect after a server crash and resume the game

Covers how disconnected players rejoin an ongoing match that was persisted on disk. In both protocols, login triggers a scan of pending restorations: if the nickname matches a saved snapshot, the player is added to the reconnected set. Once all players in the snapshot have reconnected, `MatchManager` restores the game by rebuilding the `GameManager` and `Controller`, injects the game visitor into all handlers, and launches `resume()` on a dedicated thread, which broadcasts the current board, phase, and active player to all reconnected clients.